



MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES
BSE4104: EMERGING TRENDS IN SOFTWARE ENGINEERING

AI-NATIVE & AGENTIC ENGINEERING CAPSTONE: WEEK 3 SUBMISSION

WEEK 3 PROGRESS REPORT

PROJECT: UNIVERSITY STUDENT-SUPPORT CASE AGENT

STUDENT NAMES	STUDENT NO.	REGISTRATION NO.
KABANDA ARTHUR	2300708510	23/U/08510/PS
TUMUKUNDE KATO ANDREW	2300718082	23/U/18082/PS
GARANGA JOHN	2300722975	23/X/22975/PS
ARIKO SOSSY JOEL	2300706399	23/U/06399/EVE

14-18 SEPTEMBER 2026

Week 3 Progress Report: Context Engineering and RAG

Project: University Student-Support Case Agent (BSE4104)

Week: 14-18 September 2026

Contributor: (AI Engineering / Application Integration)

Status: Completed and tested locally

1. Weekly objective

This week extended the Week 2 foundation-model baseline into a retrieval-augmented generation (RAG) system. The application now answers supported student-support questions using a controlled, traceable knowledge corpus instead of relying only on model memory. Retrieved evidence is attached to the model prompt and the API records the source documents used for each response.

2. Corpus and provenance

A controlled corpus was assembled from public Makerere sources and clearly labelled team-authored synthetic material. The source of truth is `knowledge/corpus.json`, with the generated register in `knowledge/source_register.md`.

The corpus contains 11 indexed documents: 9 real public sources and 2 synthetic documents. `src/fetch_corpus.py` downloads active sources, verifies TLS, records HTTP status, fetch time, byte count and SHA-256 checksums, and extracts text from PDF and HTML sources into `knowledge/text/`. Synthetic documents D08 and D09 are labelled both in the manifest and inside their file content. Five unavailable or scan-only sources are recorded as excluded instead of being silently omitted.

This provides traceability from a retrieved chunk back to its document, provenance, source type and extraction record. No real student data is stored in the corpus.

3. Ingestion, chunking and indexing

The ingestion and chunking pipeline is implemented in `src/fetch_corpus.py` and `src/chunker.py`.

- PDF page markers are preserved so citations can identify pages.
- Markdown headings and PDF page markers create structural sections.
- Paragraphs are packed into chunks of approximately 800 characters with 80-character overlap.
- Every chunk retains its source, title, real/synthetic kind, section heading and sequential chunk index.
- Chunk limits, ordering, metadata and page citations are covered by tests.

`src/indexer.py` embeds the chunks and stores them in a local persistent ChromaDB collection under `knowledge/index/`. Gemini embeddings are the configured provider, with an ONNX local fallback. The index can be rebuilt with `python src/indexer.py --force` and inspected with `python src/indexer.py --status`.

A compatibility issue was found when the persisted index used ONNX while runtime configuration selected Gemini. Chroma rejected the provider mismatch. The index-opening logic now reuses the persisted embedding function for an existing collection, while explicit rebuilds still use the selected provider.

4. Retrieval and model integration

`src/retriever.py` queries ChromaDB and returns source-aware chunks formatted as a RETRIEVED EVIDENCE block. Each result contains document source, title, type, section, chunk index and distance. `src/main.py` adds the evidence and the student message to the v2.0 prompt, then returns the source document IDs in the `/chat` response and CLI output.

Retrieval was improved during evaluation by:

- applying an evidence-coverage gate so unsupported requests do not receive arbitrary nearest-neighbour citations;
- filtering requests that require private records or actions not implemented yet, such as case status, grades, fee waivers, admissions, passwords and ticket execution;
- normalizing common terms such as register/registration and programme/programmes;

- searching a wider candidate pool;
- supplementing vector results with lexical metadata matches; and
- prioritizing exact title and heading matches so short documents such as D07 are not hidden by the broad D11 policy manual.

The v2.0 prompt specification defines evidence-only answering, uncertainty behaviour, citation rules and the Sources: output line. The architecture is documented in docs/architecture/WEEK-3-RAG-ARCHITECTURE.md.

5. Evaluation and failures

Fifteen RAG cases were created across answerable, partially supported, tool-dependent and unanswerable scenarios. The cases test source citation, evidence grounding, unsupported questions and synthetic-source labelling.

The final local evaluation uses a deterministic model stub so it is reproducible and does not depend on Gemini quota. All 15 RAG cases pass. The evaluation table is in docs/evaluation/WEEK-3-RAG-EVALUATION.md.

Four failures were investigated and recorded in docs/evaluation/WEEK-3-RAG-FAILURES.md:

- 1 Chroma embedding-provider mismatch between the persisted index and runtime configuration.
- 2 Stale evaluation mappings referring to excluded or unavailable sources.
- 3 False-positive evidence for unanswerable questions because nearest-neighbour retrieval had no coverage gate.
- 4 Under-ranking of short, specific sources such as D07.

The first, third and fourth issues were fixed in the implementation. The evaluation map was aligned with the active corpus. The failure catalogue retains the causes and resolutions as evidence of engineering iteration.

6. Validation evidence

The final local test run was:

```
36 passed, 1 warning
```

Relevant subsets were:

- tests/test_retrieval.py: 10 passed
- tests/test_evaluation.py: 10 passed
- tests/test_rag_evaluation.py: 15 passed

The local evaluation uses a deterministic model stub. A live Gemini run can be enabled with RAG_EVAL_LIVE=1, but earlier live runs encountered HTTP 429 free-tier quota limits. This is recorded as an environment limitation rather than reported as a passing live-model evaluation.

7. Deliverables and next step

Week 3 deliverables are present: the controlled corpus and source register, working ingestion/chunking/indexing/retrieval pipeline, architecture diagram, source-grounded model integration, 15-case evaluation results, documented grounding failures, and this progress report.

The project is ready to move to Week 4 tool integration. The next planned capability is bounded tool use for case lookup and ticket creation, with authorization and failure tests.

AI-use disclosure: Code and documentation were developed with AI assistance under contributor direction. Design decisions, testing, evaluation review and final verification were performed by the contributor.